"AI-powered Multi-Document Summarization System"

A

Project Report

submitted in partial fulfilment of the

requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING-AIML

By:

Name	USN NO	Program
LOKESHWAR REDDY	23BTRCL217	CSE(AIML-C)
P.Rahul	23BTRCL259	CSE(AIML-C)
YASHWANTH H M	23BTRCL093	CSE(AIML-C)
P,Chethan	23BTRCL227	CSE(AIML-C)
S.J. KAILASH	23BTRCL233	CSE(AIML-C)
V.Jabivulla	23BTRCL237	CSE(AIML-D)

Under the guidance of

Mentor Name: Dr.Sugumar R

Position/Post: Associate Professor

Program Name: AIML

Faculty of Engineering and Technology
Jain (Deemed to be University) Bengaluru
562112

2025-26

Department of Computer Science and Engineering
School of Computer Science & Engineering
Faculty of Engineering & Technology
Jain Global Campus, Kanakapura Taluk - 562112
Ramanagara District, Karnataka, India

CERTIFICATE

This is to certify that the project work titled **"AI-powered Multi-Document Summarization System"** is carried out by **M.Lokeshwar Reddy(23BTRCL217),P.Chethan(23BTRCL227),S.J.Kailash(23BTLCL233), V.Jabivulla(23BTRCL237),P.Rahul(23BTRCL259),Yashwanth(23BTRCL093)** bonafide student(s) of Bachelor of Technology at the School of Engineering & Technology, Faculty of Engineering & Technology, JAIN (Deemed-to-be University), Bangalore in partial fulfillment for the award of degree in Bachelor of Technology in Computer Science and Engineering, during the year **2025-2026**.

Dr.Sugumar R	**Dr. Sankar k**	**Dr. V Chandrashekar**
PCL Mentor, Computer Science and Engineering, School of Computer Science & Engineering, Faculty of Engineering & Technology JAIN (Deemed to-be University)	PCL coordinator, Computer Science and Engineering, School of Computer Science & Engineering, Faculty of Engineering & Technology JAIN (Deemed to-be University)	Head of department, Computer Science and Engineering, School of Computer Science & Engineering, Faculty of Engineering & Technology JAIN (Deemed to-be University)

Name of the Examiner Signature of Examiner

1.

2.

DECLARATION

We, **LOKESHWAR REDDY (23BTRCL217), P. RAHUL (23BTRCL259), YASHWANTH H M (23BTRCL093), P. CHETHAN (23BTRCL227), S. J. KAILASH (23BTRCL233), V. JABIVULLA (23BTRCL237)**, students of **B.Tech in Computer Science and Engineering (Artificial Intelligence and Machine Learning)** at the School of Engineering & Technology, Faculty of Engineering & Technology, **JAIN (Deemed-to-be University), Bangalore**, hereby declare that the project work titled **"AI-Powered Multi-Document Summarization System"** has been carried out by us and submitted in partial fulfillment for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (AIML)** during the academic year **2023–2027**.

Further, we declare that the work presented in this project has not been submitted previously by any person for the award of any degree or diploma in any University or Institution, to the best of our knowledge and belief.

Name	USN NO	SIGNATURE
LOKESHWAR REDDY	23BTRCL217	
P.Rahul	23BTRCL259	
YASHWANTH H M	23BTRCL093	
P,Chethan	23BTRCL227	
S.J. KAILASH	23BTRCL233	
V.Jabivulla	23BTRCL237	

ACKNOWLEDGEMENT

It gives us great pleasure to acknowledge the assistance and support of many individuals who have contributed to the successful completion of this project.

First, we take this opportunity to express our sincere gratitude to the Faculty of Engineering & Technology, JAIN (Deemed-to-be University), Bangalore, for providing us with the opportunity to pursue our Bachelor's degree in this esteemed institution.

We are deeply thankful to several individuals whose invaluable contributions have made this project a reality. We would like to extend our heartfelt gratitude to **Dr. Chandraj Roy Chand, Chancellor**, for his continuous efforts in promoting excellence in education and research at Jain (Deemed-to-be University). We are also sincerely grateful to **Dr. Raj Singh, Vice Chancellor**, and **Dr. Dinesh Nilkant, Pro Vice Chancellor**, for their constant encouragement and support.

We express our sincere thanks to **Dr. Jitendra Kumar Mishra, Registrar**, for his guidance and support, which have been instrumental in shaping our academic journey.

It is a matter of great privilege to express our sincere thanks to **Dr. V. Chandrashekhar, Program Head of Computer Science and Engineering (Artificial Intelligence and Machine Learning)**, for providing valuable academic guidance and support throughout the project.

We would like to express our heartfelt gratitude to our **Project Coordinator, Dr. Sankar K and Mentor, Dr.Sugumar R** for his constant guidance, encouragement, and valuable suggestions during the course of this project.

We also extend our thanks to all the faculty members and staff of the Department of Computer Science and Engineering for their support and cooperation.

Finally, we express our gratitude to our family and friends for their continuous support, motivation, and encouragement throughout this work.

We would like to thank everyone who has directly or indirectly contributed to the successful completion of this project.

Signature of Student(s)

TABLE OF CONTENTS

List of Content	Page No.
1. ABSTRACT	7-8
2. INTRODUCTION	10–13
2.1 Background of AI-Based Summarization	10
2.2 Problem Statement	11
2.3 Objectives of the Project	12
2.4 Scope of the System	12
3. LITERATURE REVIEW	13-17
3.1 Overview of Text Summarization Techniques	13
3.2 Extractive vs Abstractive Summarization	14
3.3 Transformer Models (BART, T5, PEGASUS)	15
3.4 Emergence of Modern LLMs	16
3.5 Efficiency Improvements in Recent Models	16
3.6 Limitations of Existing Systems	17
4. SYSTEM ANALYSIS	18–22
4.1 Existing System	18
4.2 Proposed System	19
4.3 Feasibility Study	21
5. SYSTEM ARCHITECTURE	23–25
5.1 Overall Architecture Diagram	23
5.2 Client-Server Architecture	24
5.3 AI Integration Layer	24
5.4 Data Flow Overview	24
6. CORE FEATURES OF THE SYSTEM	26–28
6.1 Multimodal Document Processing	26
6.2 Advanced Summarization Engine	27
6.3 Interactive Features	28
6.4 User Management and History	28
7. TECHNOLOGY STACK	29–31
7.1 Programming Languages	29
7.2 Frameworks and Libraries	29
7.3 AI Models and APIs	30
7.4 Database Technologies	31
7.5 Frontend Technologies	31
8. SYSTEM DESIGN	32–36
8.1 Data Flow Diagram (DFD)	32
8.2 Flowchart of System Workflow	32

8.3 Use Case Diagram	34
8.4 Database Design (ER Diagram)	35
8.5 Sequence Diagram	36
9. IMPLEMENTATION	37–39
9.1 Frontend Implementation	37
9.2 Backend Implementation	37
9.3 File Processing Modules	38
9.4 AI Model Integration	39
9.5 API Integration	39
10. WORKING OF THE SYSTEM	40
11. SECURITY AND PERFORMANCE	41-42
11.1 Secure File Handling	41
11.2 Password Encryption	41
11.3 API Rate Limit Handling	42
11.4 Performance Optimization	42
12. RESULTS AND DISCUSSION	43–45
13. TESTING	46-47
14. ADVANTAGES AND LIMITATIONS	48
15. APPLICATIONS OF THE SYSTEM	48
16. FUTURE ENHANCEMENTS	48–50
17. CONCLUSION	51
18. REFERENCES / BIBLIOGRAPHY	52
19. APPENDICES	53-54

1.ABSTRACT

The exponential growth of digital information in recent years has significantly increased the demand for efficient tools capable of processing, analyzing, and summarizing large volumes of textual data. Individuals and organizations are often overwhelmed by the sheer amount of content available across various formats such as research papers, reports, presentations, and multimedia sources. Manually extracting meaningful insights from such data is not only time-consuming but also cognitively demanding. To address this challenge, this project proposes the development of an AI-Powered Multi-Document Summarization Application, a robust and scalable web-based system designed to automate information extraction and enhance knowledge consumption.

This application leverages the power of state-of-the-art Large Language Models (LLMs), particularly the Google Gemini Generative AI models (Gemini 1.5 and Gemini 2.0 Flash/Pro), which are known for their high efficiency, contextual understanding, and low-latency performance. These models enable the system to generate highly coherent, context-aware, and semantically rich summaries. Unlike conventional extractive summarization techniques that simply select key sentences from the original text, the proposed system utilizes abstractive summarization, allowing it to rephrase, condense, and synthesize information in a human-like manner.

A key strength of the application lies in its ability to handle multi-format and multi-source inputs. Users can upload documents in various formats including PDFs, PowerPoint presentations (PPT/PPTX), plain text files, and even provide web URLs for dynamic content extraction. Additionally, the system integrates functionality to automatically retrieve and process YouTube video transcripts, thereby extending its capabilities beyond static documents to multimedia content. This ensures a unified platform for summarizing diverse types of information.

The system also introduces multimodal processing capabilities, which enhance the quality and usability of the generated summaries. It intelligently extracts relevant textual content along with embedded images from PDFs and PPTs, filters out irrelevant visuals, and incorporates meaningful images into the summary output. This integration of visual elements helps users better understand complex concepts and improves overall comprehension.

To accommodate different user requirements, the summarization engine supports multiple output formats and customization options. Users can choose from:

- Detailed paragraph summaries for in-depth understanding
- Concise bullet-point summaries for quick review
- Structured academic formats suitable for reports and research

- Mermaid.js-based flowcharts to visualize processes and relationships
- Mindmaps to represent hierarchical and conceptual connections

Beyond summarization, the system enhances interactivity through a Document-Based Question and Answering (Q&A) module. This feature allows users to ask questions related to the uploaded content, and the system generates accurate responses strictly grounded in the provided documents, ensuring reliability and relevance. Furthermore, the integration of Edge-TTS (Text-to-Speech) technology enables the conversion of textual summaries into high-quality audio, making the application more accessible and user-friendly, especially for auditory learners.

From a system design perspective, the application is built on a Python Flask backend, which ensures modularity, scalability, and efficient request handling. User authentication and session management are securely handled using Flask-Login, while an SQLite database is employed to maintain user data, document history, and generated summaries. The frontend interface is designed to be intuitive and responsive, allowing seamless interaction between users and the AI engine.

In addition, the system incorporates error handling, file validation, and efficient preprocessing techniques to ensure robustness and reliability. Advanced parsing libraries are used for extracting text from PDFs and PPTs, while Natural Language Processing (NLP) techniques are applied for cleaning, tokenization, and preparing the input data before feeding it into the LLM.

Overall, this project demonstrates a comprehensive application of Generative AI in real-world problem solving, combining advanced NLP techniques, multimodal processing, and interactive features into a unified platform. By significantly reducing reading time, improving comprehension, and enabling intelligent information retrieval, the system provides a powerful tool for students, researchers, educators, and professionals to efficiently manage and understand large volumes of information.

LIST OF FIGURES

Figure No.	Title of Figure	Page No.
Figure 2.1	Information Overload and Text Summarization Concept	10
Figure 3.1	Extractive vs Abstractive Summarization Comparison	14
Figure 3.2	Transformer Model Architecture	15
Figure 4.1	Existing System Workflow	18
Figure 4.2	Proposed System Architecture Overview	20
Figure 5.1	Overall System Architecture of AI-Powered Summarization Application	23
Figure 5.4	Data Flow Overview : AI-Powered Summarizer Lifecycle	25
Figure 7.1	Technology Stack Of the System	30
Figure 8.1	Level 0 Data Flow Diagram (DFD)	32
Figure 8.2	Flow Diagram	33
Figure 8.3	System Workflow Flowchart	34
Figure 8.4	Use Case Diagram of the System	35
Figure 8.5	Entity Relationship (ER) Diagram	36
Figure 10.1	End-to-End System Working Pipeline	40
Figure 12.1	User Login Interface	43
Figure 12.2	Dashboard Interface	43
Figure 12.3	File Upload Interface	44
Figure 12.4	Generated Summary Output	44
Figure 12.5	Document Chat (Q&A) Interface	45

2.INTRODUCTION

2.1 Background of AI-Based Summarization

In the modern digital era, the rapid expansion of information has created both opportunities and challenges. With the widespread use of the internet, cloud platforms, and digital publishing, enormous amounts of data are generated every second in the form of academic research papers, business analytics reports, online articles, blogs, and multimedia content. While this abundance of information is valuable, it has also led to a problem commonly referred to as information overload. Individuals often struggle to process, analyze, and retain relevant information efficiently due to time constraints and cognitive limitations.

Text summarization has emerged as a crucial solution to this problem. The primary goal of summarization is to reduce lengthy content into a shorter version while preserving its key ideas and meaning. Traditionally, summarization techniques were based on extractive approaches, where important sentences were selected directly from the original text based on statistical features such as frequency, position, or keyword importance. Although effective to some extent, extractive methods often produce summaries that lack coherence and fail to capture the deeper context of the content.

Figure 2.1: Information Overload and Text Summarization Concept

With the evolution of Artificial Intelligence (AI) and Natural Language Processing (NLP), more advanced techniques have been developed. In particular, the rise of Large Language Models (LLMs) has revolutionized the field of text summarization. Models such as Google Gemini are capable of performing abstractive summarization, where the system understands the context of the input and generates new sentences that convey the meaning in a more natural and human-like way. These models are trained on massive datasets and can handle complex language structures, making them highly effective for real-world applications.

Furthermore, modern summarization is no longer limited to plain text. With the increasing use of presentations, visual content, and multimedia platforms, there is a growing need for multimodal summarization systems. Such systems are capable of processing not only text but also images, diagrams, and even video transcripts. This project builds upon these advancements to develop a next-generation summarization application that integrates AI, multimodal processing, and interactive features into a single platform.

2.2 Problem Statement

Despite significant advancements in summarization technologies, existing tools still face several limitations that reduce their effectiveness in practical scenarios. Many currently available summarization systems are designed with narrow functionality and lack flexibility, which creates challenges for users dealing with diverse content formats.

One of the major issues is the limited support for input formats. Most traditional summarization tools are designed to process only plain text, requiring users to manually extract content from complex file formats such as PDFs or PowerPoint presentations. This manual process is time-consuming and prone to errors, especially when dealing with large documents or multiple files.

Another critical limitation is the loss of visual context. Many documents, particularly academic and technical ones, contain important visual elements such as graphs, charts, diagrams, and images. These elements play a vital role in conveying information, but most summarization tools ignore them completely. As a result, the generated summaries may lack essential context, leading to incomplete understanding.

In addition, existing tools often provide rigid and generic output formats. Different users have different requirements depending on their use case. For example, a student may prefer bullet points for quick revision, while a professional may require structured summaries or flowcharts for presentations. However, most tools only generate paragraph-based summaries, limiting their usability.

Another challenge is the lack of interactivity. Traditional summarization systems produce static outputs, which do not allow users to explore the content further. Users often need to ask questions or clarify specific points within the document, but current tools do not provide such capabilities. Similarly, accessibility features such as audio summaries are rarely integrated.

Considering these challenges, there is a clear need for a comprehensive and intelligent system that can handle multiple formats, preserve visual context, provide customizable outputs, and support interactive user engagement. This project aims to address these gaps by developing a unified AI-powered summarization platform.

2.3 Objectives of the Project

The main objective of this project is to design and develop a smart, efficient, and user-friendly web application that enhances the way users interact with large volumes of information. The system aims to provide accurate, flexible, and context-aware summaries using advanced AI technologies.

One of the primary objectives is to enable multi-format input processing. The system should be capable of handling various types of input data, including plain text, PDF documents, PowerPoint presentations, and web URLs. Additionally, it should support the extraction of transcripts from YouTube videos, allowing users to summarize multimedia content seamlessly.

Another important objective is to implement multimodal content extraction. The system should not only extract textual information but also identify and retain relevant images and diagrams from documents. This ensures that the generated summaries are more informative and visually enriched.

The project also focuses on providing customizable output formats. Users should have the flexibility to choose the format of the summary based on their needs. These formats include detailed paragraphs, concise bullet points, structured academic summaries, flowcharts generated using Mermaid.js, and mindmaps for conceptual understanding.

To enhance user experience, the system aims to include interactive features such as a Document-Based Question and Answer (Q&A) module. This allows users to ask questions related to the uploaded content and receive precise answers. In addition, the integration of a Text-to-Speech (TTS) feature enables the conversion of summaries into audio format, improving accessibility.

Another key objective is to ensure security and data management. The system should implement secure user authentication and maintain a database to store user history, uploaded files, and generated summaries. This helps in providing a personalized and efficient user experience.

2.4 Scope of the System

The scope of this project includes the complete development and deployment of a web-based client-server application for intelligent document summarization. The system is designed to be scalable, efficient, and capable of handling real-world use cases.

On the backend, the application is built using Python and the Flask framework, which handles server-side logic, API routing, and request processing. The backend is responsible for managing file uploads, extracting content from different formats, and interacting with the Gemini API for generating summaries. It also ensures secure handling of user data and temporary files.

The system integrates with the Google Generative AI (Gemini) API, which serves as the core engine for summarization and natural language understanding. The integration layer manages API communication, prompt engineering, and response processing to ensure high-quality outputs.

The frontend of the application is developed using HTML, CSS, and JavaScript, providing a responsive and user-friendly interface. It includes features such as file upload options, summary display panels, interactive chat interfaces, and visualization tools. The frontend also supports rendering of advanced formats such as Markdown, LaTeX, and Mermaid.js diagrams.

The system is designed to handle documents up to a specific size (e.g., 64MB) and a limited number of slides in presentations to ensure optimal performance. It also includes error handling mechanisms, input validation, and efficient preprocessing techniques to maintain reliability.

Additionally, the application supports database management using SQLite, which stores user credentials, session data, and history of generated summaries. This allows users to revisit previously processed documents and improves overall usability.

3.Literature Review

3.1 Overview of Text Summarization Techniques

Text summarization is an important area within the field of Natural Language Processing (NLP) that focuses on reducing large volumes of text into shorter, meaningful summaries while preserving essential information. With the rapid growth of digital data, the need for efficient summarization techniques has become increasingly important in domains such as education, research, business analytics, and media.

In the early stages, summarization techniques were primarily based on statistical and rule-based approaches. Methods such as Term Frequency-Inverse Document Frequency (TF-IDF) were widely used to identify important words in a document. Similarly, graph-based algorithms like TextRank treated sentences as nodes and ranked them based on their relationships, helping in identifying the most relevant sentences.

While these methods were simple and computationally efficient, they had significant limitations. They relied heavily on surface-level features such as word frequency and sentence position, without truly understanding the meaning or

context of the text. As a result, the generated summaries often lacked coherence and failed to capture the deeper semantics of the content.

The introduction of machine learning and deep learning techniques marked a major turning point in text summarization. Neural networks enabled models to learn patterns from large datasets, allowing them to better understand language structure and context. Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) models were initially used for sequence modeling tasks, including summarization. However, these models struggled with long-range dependencies and computational efficiency.

This led to the development of more advanced architectures, particularly the Transformer model, which revolutionized NLP and significantly improved summarization performance.

3.2 Extractive vs Abstractive Summarization

Text summarization techniques can be broadly classified into two categories: extractive summarization and abstractive summarization.

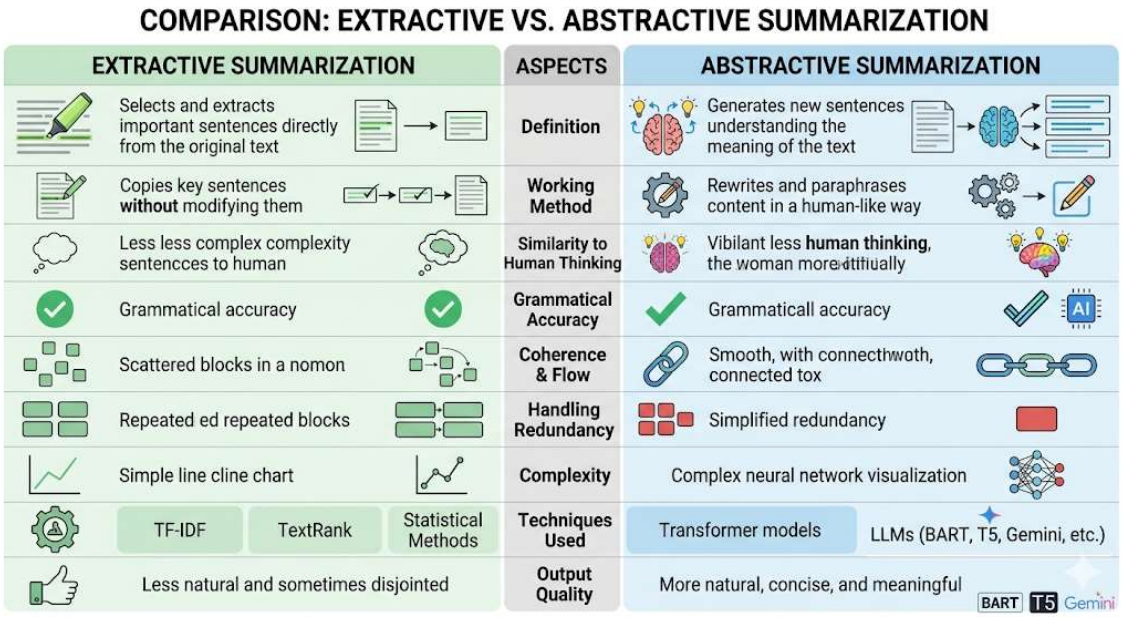


Figure 3.1: Extractive vs Abstractive Summarization Comparison

Despite its advantages, abstractive summarization is more challenging to implement because it requires a deep understanding of language and context. However, with the rise of deep learning and transformer-based models, abstractive summarization has become the dominant approach in modern systems, including the one proposed in this project.

3.3 Transformer Models (BART, T5, PEGASUS)

The introduction of the Transformer architecture in 2017 brought a major breakthrough in NLP. Unlike previous models, Transformers use a mechanism called self-attention, which allows them to focus on different parts of the input text simultaneously. This enables better understanding of long-range dependencies and contextual relationships within the text.

Several powerful transformer-based models have been developed specifically for text generation and summarization tasks:

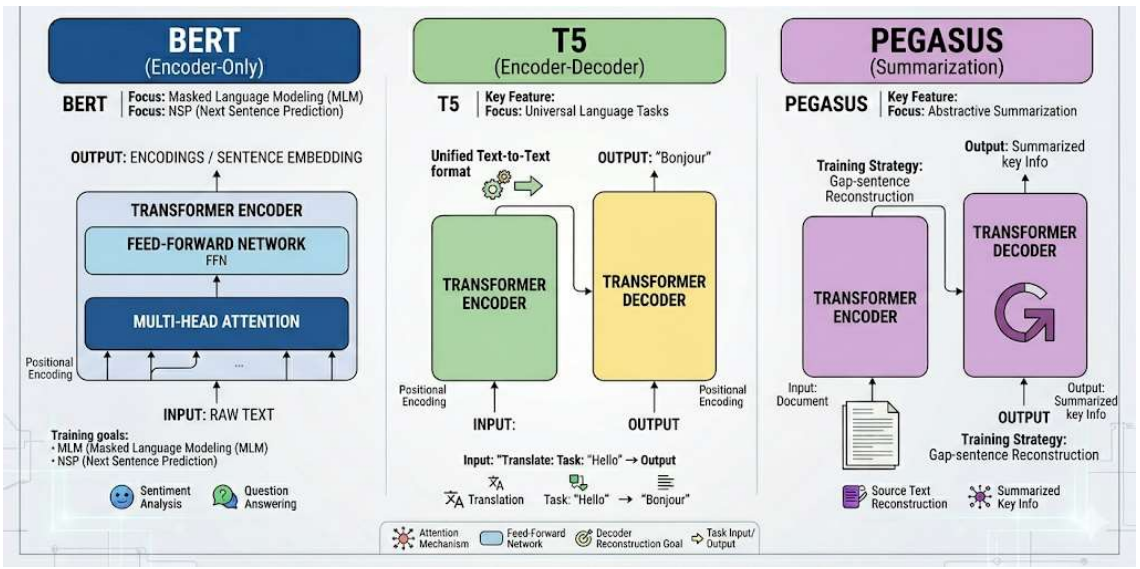


Figure 3.2: Transformer Model Architecture

BART (Bidirectional and Auto-Regressive Transformers)

BART combines the strengths of bidirectional and autoregressive models. It uses an encoder-decoder architecture, where the encoder reads and understands the input text, and the decoder generates the summary. BART is particularly effective for tasks that require both comprehension and generation, such as summarization.

T5 (Text-to-Text Transfer Transformer)

T5 introduced a unified framework where all NLP tasks are treated as text-to-text problems. This means that both the input and output are represented as text strings. This flexibility allows T5 to perform a wide range of tasks, including summarization, translation, and question answering, using the same model architecture.

PEGASUS

PEGASUS is specifically designed for summarization tasks. During training, it uses a unique technique called Gap Sentence Generation (GSG), where important sentences are removed from the input and the model learns to predict them. This makes PEGASUS highly effective in generating high-quality summaries, especially for long documents.

These models significantly improved the quality of abstractive summarization, making it more accurate, coherent, and context-aware.

3.4 Emergence of Modern Large Language Models (LLMs)

The field of NLP has undergone another major transformation with the development of Large Language Models (LLMs). Models such as OpenAI's GPT series, Anthropic's Claude, and Google's Gemini have introduced a new level of intelligence and flexibility in language processing.

Unlike earlier models that required task-specific training, LLMs are trained on massive datasets containing diverse text from the internet. This allows them to perform tasks such as summarization, translation, and question answering with minimal or no additional training, a capability known as zero-shot learning.

One of the key advantages of LLMs is their ability to:

- Understand complex context and semantics
- Generate human-like responses
- Adapt to different domains and formats
- Handle long and detailed documents

Google's Gemini models stand out due to their multimodal capabilities, meaning they can process not only text but also images, audio, and other forms of data. This makes them highly suitable for applications involving complex documents such as PDFs and presentations that include both textual and visual information.

3.5 Efficiency Improvements in Recent Models

Although LLMs are powerful, they traditionally required significant computational resources, making them expensive and slow for real-time applications. To address these challenges, recent research has focused on improving efficiency through techniques such as:

- **Model distillation:** Reducing model size while retaining performance

- **Quantization:** Lowering precision to reduce computational cost
- **Optimized architectures:** Designing models for faster inference

Google's Gemini 1.5 and 2.0 Flash models represent a major advancement in this area. These models are specifically optimized for low latency and high throughput, making them suitable for real-time applications such as chat interfaces and live summarization tools.

The Flash models offer:

- Faster response times
- Reduced API costs
- High accuracy comparable to larger models

This makes them ideal for building scalable applications like the one proposed in this project, where speed and efficiency are critical.

3.6 Limitations of Existing Systems

Despite the rapid advancements in summarization technologies, many existing systems still face several limitations that affect their usability and performance.

One of the major limitations is single-modality processing. Most tools are designed to handle only plain text and ignore other important elements such as images, charts, and diagrams. This results in summaries that lack important contextual information.

Another issue is the lack of flexible output formats. Many tools generate only basic paragraph summaries and do not support structured outputs such as flowcharts, mindmaps, or formatted reports. This limits their usefulness for different types of users.

Additionally, although modern LLMs support large context windows, many applications fail to utilize them effectively. This leads to incomplete analysis of long documents, where only a portion of the content is considered.

There is also a lack of integration of advanced features. Users often need to rely on multiple tools for tasks such as text extraction, summarization, question answering, and text-to-speech conversion. This creates a fragmented workflow and reduces overall efficiency.

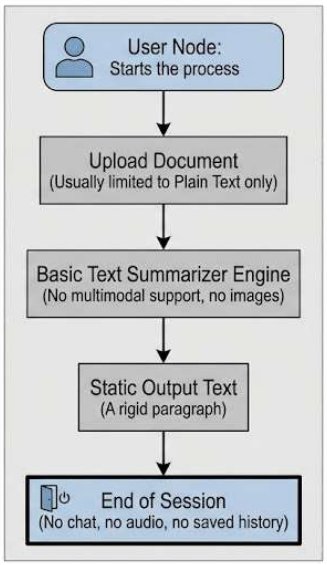
4. System Analysis

4.1 Existing System

In the current digital environment, several tools are available for document summarization. These include online summarizer websites, built-in PDF readers, and basic NLP-based applications. However, most of these systems are designed with limited functionality and focus only on specific types of input, primarily plain text. As a result, they fail to meet the needs of users dealing with diverse and complex data formats.

Most existing systems follow a single-purpose architecture, where each tool performs only one function. For example, one tool may extract text from PDFs, another may summarize text, and yet another may convert text to speech. This fragmented approach requires users to switch between multiple platforms, leading to inefficiency and increased effort.

Figure 4.1: Existing System Workflow



Additionally, these systems lack the ability to process multimodal data, which includes not only text but also images, diagrams, and multimedia content. In modern documents such as research papers and presentations, visual elements play a crucial role in conveying information. Ignoring these elements results in incomplete summaries and reduced understanding.

Another major limitation is the lack of flexibility in output formats. Most tools generate summaries in plain paragraph form, without offering options for structured or visual representations such as flowcharts or mindmaps. This reduces their usefulness for different types of users and applications.

Furthermore, existing systems are largely non-interactive. Once a summary is generated, users cannot interact with it further. There is no option to ask questions, clarify doubts, or explore specific sections of the content. Similarly, accessibility features like text-to-speech are often missing.

Drawbacks of Existing System

Issue	Description	Impact on Users
Lack of Multimodal Support	Cannot process images, charts, or diagrams	Loss of important context
Limited Input Formats	Mostly supports only plain text	Manual effort required for file conversion
Static Outputs	Provides only paragraph summaries	Not suitable for all use cases
No Interactivity	No Q&A or user engagement	Reduced understanding
No Video Support	Cannot directly summarize YouTube/web content	Extra tools required
Fragmented Tools	Requires multiple applications	Time-consuming workflow

4.2 Proposed System

To overcome the limitations of existing systems, this project proposes an AI-Powered Multi-Document Summarization Application. The system is designed as a centralized and intelligent platform that integrates multiple functionalities into a single application.

The proposed system leverages advanced Large Language Models (LLMs), specifically Google Gemini, to perform high-quality abstractive summarization. It is capable of handling multiple input formats, including PDFs, PowerPoint presentations, text files, and web URLs, making it highly versatile.

One of the key features of the system is its multimodal extraction capability. Unlike traditional tools, it not only extracts textual content but also identifies and retains relevant images and diagrams. This ensures that the generated summaries provide both textual and visual understanding.

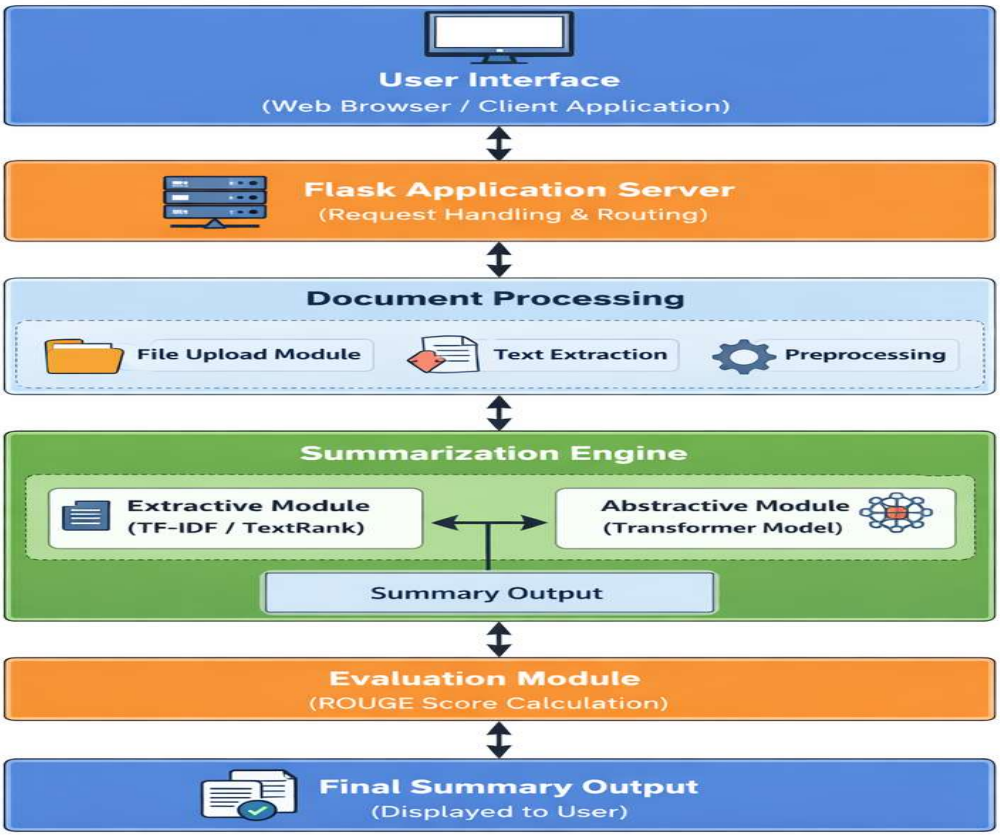


Figure 4.2: Proposed System Architecture Overview

The system also offers dynamic output formatting, allowing users to choose how they want the summary to be presented. This includes paragraph summaries, bullet points, flowcharts using Mermaid.js, and structured mindmaps. Such flexibility makes the system suitable for various applications, from academic study to business presentations.

Another important feature is interactivity. The system includes a Document-Based Question and Answer (Q&A) module, enabling users to interact with the content and gain deeper insights. Additionally, the integration of Text-to-Speech (TTS) allows users to listen to summaries, improving accessibility.

The system also supports automatic content extraction from web URLs and YouTube videos, eliminating the need for external tools. Furthermore, it includes a secure database system for managing user sessions and history.

Advantages of Proposed System

Feature	Description	Benefit
Multimodal Processing	Extracts text and images	Better understanding
Multi-format Support	PDF, PPT, Text, URLs, YouTube	High flexibility
Dynamic Output	Paragraphs, bullets, flowcharts, mindmaps	Customizable summaries
Interactive Q&A	Ask questions on documents	Deeper insights
Text-to-Speech	Converts text to audio	Improved accessibility
Centralized Platform	All features in one system	Saves time and effort
Session Management	Stores user history	Easy retrieval

4.3 Feasibility Study

A feasibility study is conducted to evaluate whether the proposed system is practical and achievable. It ensures that the project can be successfully implemented within available resources and constraints.

1. Technical Feasibility

The system is highly feasible from a technical perspective. It is built using modern and reliable technologies such as Python and Flask, which are widely used for web application development. The integration with the Google Gemini API ensures access to powerful AI capabilities without the need for building complex models from scratch.

Supporting libraries such as:

- PyMuPDF (for PDF processing)
- python-pptx (for PowerPoint extraction)
- youtube-transcript-api (for video transcripts)

are stable, well-documented, and easy to integrate. The frontend technologies (HTML, CSS, JavaScript) are standard and ensure smooth user interaction. Therefore, the system can be implemented efficiently without major technical challenges.

2. Economic Feasibility

The project is economically viable as it relies largely on open-source tools and frameworks. Technologies like Flask, SQLite, and BeautifulSoup are freely available, reducing development costs.

The primary cost component is the use of AI APIs. However, by utilizing efficient models such as Gemini Flash, the system minimizes operational costs. Additionally, proper API usage strategies, such as request optimization and fallback mechanisms, further reduce expenses.

Overall, the system can be developed and maintained at a low cost, making it suitable for both academic and real-world deployment.

3. Operational Feasibility

The system is designed with a strong focus on usability and user experience. It provides a simple and intuitive interface where users can easily upload documents or paste links to generate summaries.

Key operational advantages include:

- Minimal user effort (simple upload and click)
- Automated background processing
- Fast response time
- Easy navigation and interaction

The system also ensures proper error handling and smooth operation, making it reliable for daily use.

Feasibility Summary Table

Feasibility Type	Evaluation	Conclusion
Technical	Uses proven technologies and APIs	Highly feasible
Economic	Low cost using open-source tools	Cost-effective
Operational	Easy to use and efficient	User-friendly

5. System Architecture

5.1 Overall Architecture Diagram

The architecture of the AI-Powered Multi-Document Summarization Application is designed with a modular and layered approach, ensuring scalability, maintainability, and efficient processing. The system is divided into three major layers:

1. Frontend Layer (User Interface)
2. Backend Layer (Application Logic)
3. AI Processing Layer (External Services)

This separation of concerns allows each component to function independently while maintaining seamless communication across the system.

The architecture diagram illustrates how user requests flow through different modules, from input submission to final output delivery. The user interacts with the frontend interface, which communicates with the Flask backend through HTTP requests. The backend processes the input, interacts with external services such as the Gemini API and TTS engine, and returns the final result.

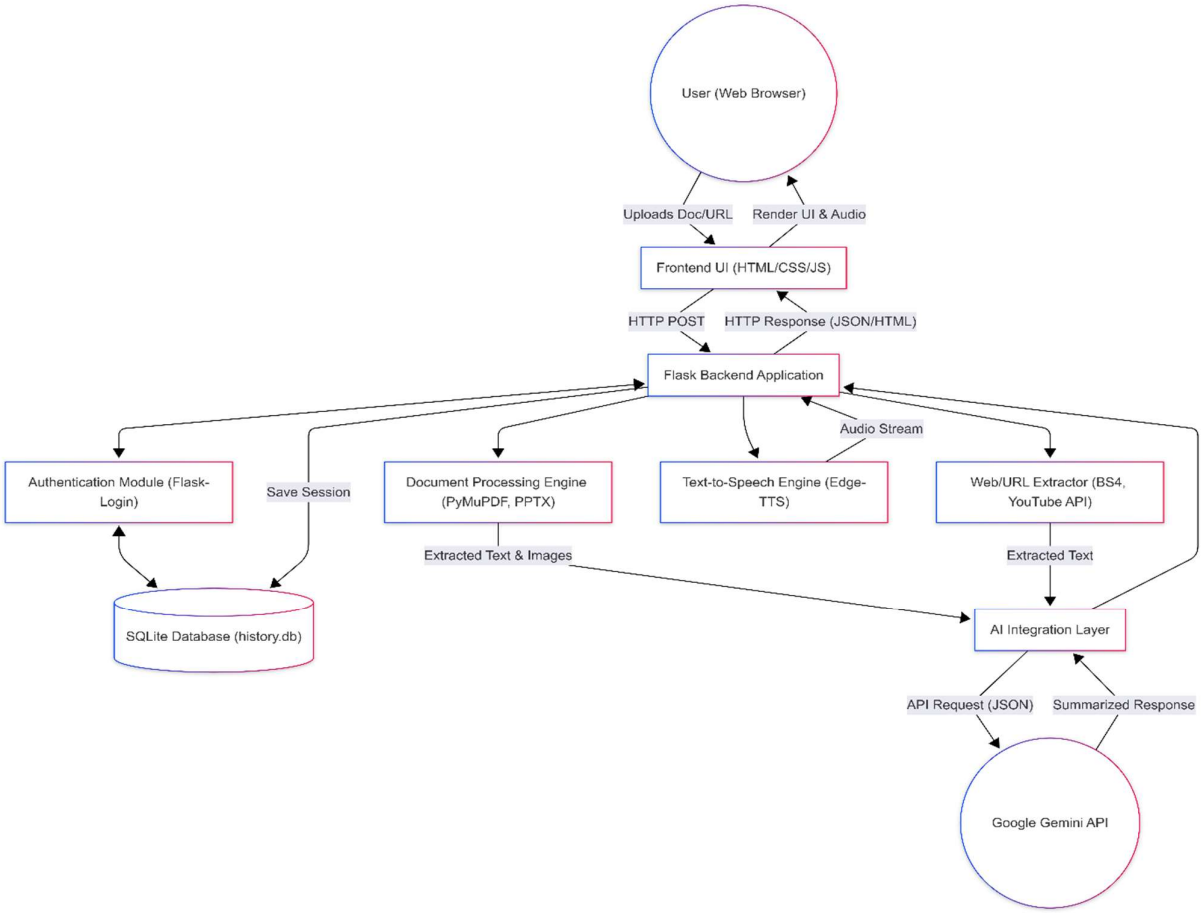


Figure 5.1: Overall System Architecture of AI-Powered Summarization Application

5.2 Client-Server Architecture

The system follows a standard Client-Server architecture pattern:

- **Client (Frontend):** The client is a web browser rendering standard HTML, customized with CSS for a responsive layout, and Vanilla JavaScript for asynchronous HTTP interactions (AJAX). The client is responsible for capturing user input (files, URLs, modes), rendering complex outputs (such as parsing Markdown and executing Mermaid.js for flowcharts), and providing interactive audio playback and chat boxes without requiring full page reloads.
- **Server (Backend):**The server is powered by Python and the Flask framework. It acts as the central orchestrator. When a request hits the server (e.g., `/summarize`), the Flask routing engine securely saves the uploaded file to a temporary directory (`tempfile.mkdtemp()`), passes the file path to the appropriate extraction script (`extract_text_and_images_from_pdf`, etc.), and then securely deletes the file from disk to preserve space and maintain privacy.

5.3 AI Integration Layer (Gemini API)

The core intelligence of the system resides in the AI Integration Layer, utilizing the `google-generativeai` SDK.

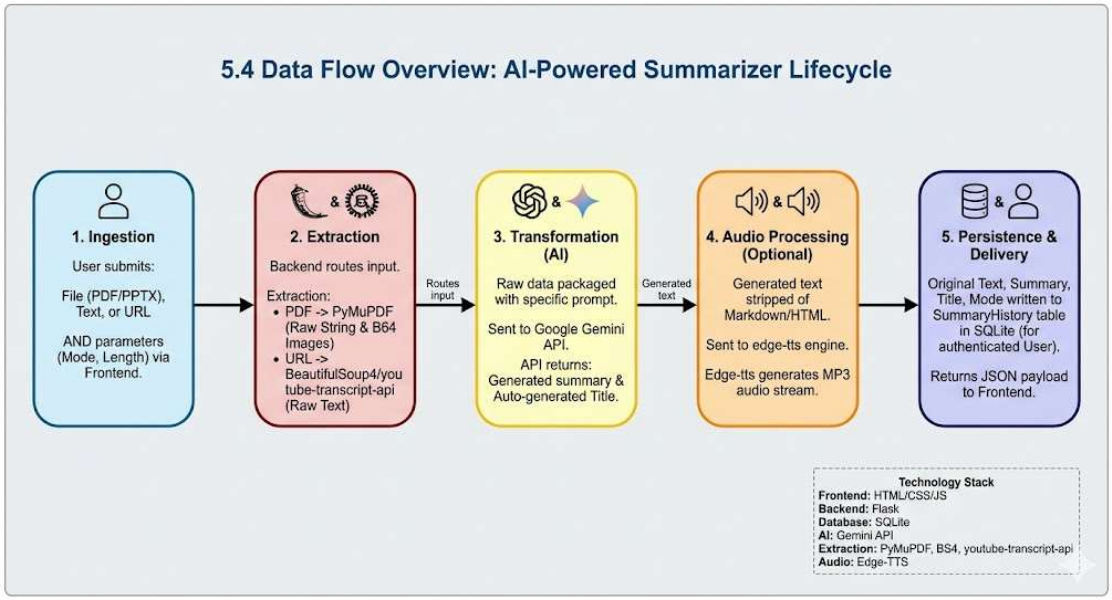
- **Prompt Engineering Orchestration:** Instead of passing raw text, the server wraps the extracted text in highly tailored, mode-specific prompt templates (e.g., `mode == 'bullets'`, `mode == 'smart_flowchart'`). These prompts enforce strict rules, such as LaTeX mathematical accuracy and Mermaid syntax constraints.
- **Multimodal Embedding:**If images are extracted from a PDF/PPT, they are Base64 encoded and passed directly to the Gemini API as `mime_type` objects alongside the text. The prompt instructs the model to place `[IMAGE_N]` markers in the summary where the image contextually belongs.
- **Robust Fallback Mechanism:**To handle API rate limits (HTTP 429 Quota Exceeded), the layer implements an automated failover loop. It iterates through an array of models (`gemini-2.5-flash`, `gemini-2.0-flash`, `gemini-flash-lite`, etc.) until a successful response is generated, ensuring high availability.

5.4 Data Flow Overview

The data lifecycle within the application is strictly linear and stateless, with persistence occurring only at the final step:

1. **Ingestion:** User submits input (File, Text, or URL) and configuration parameters (Summarization Mode, Length) via the Frontend.

2. **Extraction:** Backend routes the input. If it's a PDF, `PyMuPDF` extracts the raw string and Base64 images. If it's a URL, `BeautifulSoup4` or `youtube-transcript-api` pulls the raw text.



3. **Transformation (AI):** The raw data is packaged with the specific prompt and sent to the Gemini API. The API returns the generated summary string and an auto-generated title.

4. **Audio Processing (Optional):** The generated text is stripped of markdown/HTML and sent to the `edge-tts` engine to generate an MP3 audio stream.

5. **Persistence & Delivery:** The original text, the summary, the title, and the chosen mode are written to the `SummaryHistory` table in the SQLite database linked to the authenticated `User`. Finally, a JSON payload is returned to the Frontend for rendering.

6.CORE FEATURES OF THE SYSTEM

6.1 Multimodal Document Processing

The system's foundational capability is its ability to extract data from multiple complex file formats, capturing both textual data and visual context.

6.1.1 PDF Processing and Image Extraction

PDFs are processed using the `PyMuPDF` (fitz) library. The system iterates through every page to extract text. Simultaneously, it scans for embedded image objects. To ensure only meaningful images (such as charts or diagrams) are passed to the AI, the system employs an intelligent filtering mechanism:

- It checks image dimensions, automatically discarding tiny icons (e.g., width or height < 250px).
- It calculates the aspect ratio, discarding extreme shapes like thin horizontal dividers.
- It evaluates file size, discarding elements under 15KB.

The surviving, high-quality images are converted into Base64 encoded strings and temporarily stored for the AI integration layer.

6.1.2 PowerPoint Processing

PowerPoint files (PPT/PPTX) are parsed using `python-pptx`. The system iterates through up to 50 slides, extracting textual shape elements while maintaining slide-by-slide context (e.g., "Slide 1: [Text]"). Similar to the PDF processor, it scans shapes for embedded blobs (images). It filters out small elements based on English Metric Units (EMUs) and filesize, encoding the relevant slide graphics into Base64 for multimodal analysis.

6.1.3 URL and YouTube Content Extraction

For web-based content, the system bypasses file uploads entirely.

- **Web URLs:** It uses the `requests` library to fetch HTML and `BeautifulSoup4` to parse it. It strips out non-content tags (like `<nav>`, `<footer>`, `<script>`) to extract clean, readable text.
- **YouTube:** f the URL belongs to YouTube, the system parses the video ID and uses `youtube-transcript-api` to fetch the auto-generated or manual subtitles, seamlessly converting video spoken-word into text for summarization without downloading massive video files.

6.2 Advanced Summarization Engine

At the heart of the application is a highly dynamic prompt-engineering engine powered by Google Gemini, capable of manipulating extracted text into multiple distinct output formats based on user preference.

6.2.1 Paragraph and Bullet Summarization

These are the standard modes for academic and professional reading. The system uses specific prompts requiring the AI to format side-headings in bold and use standard dashes for bullets. Crucially, the engine has strict rules for **Mathematical Accuracy**, enforcing the use of standard LaTeX delimiters (`$ equation $` and `$$ equation $$`) so complex formulas and chemical equations from the source text are rendered perfectly in the browser.

6.2.2 Flowchart and Smart Flowchart Generation

Instead of reading text, users can visualize processes.

- **Flowchart Mode:** Converts logical processes described in the text into sequential, top-down `Mermaid.js` graphs.
- **Smart Flowchart Mode:**Focuses on hierarchical taxonomies and parent-child relationships, drawing complex multi-branch diagrams. The prompt strictly instructs the AI not to hallucinate data and to use descriptive node names while forbidding text on the connector arrows, ensuring the `Mermaid` syntax compiles flawlessly.

6.2.3 Mindmap Generation

For brainstorming and high-level overviews, the Mindmap mode instructs the AI to bypass markdown entirely and output a strict JSON structure containing a "main_topic" and an array of "subtopics" with key points. The frontend parses this JSON to draw an interactive, visual mindmap using HTML/CSS.

6.2.4 Auto Title Generation

To improve the user experience, an asynchronous AI call generates a highly relevant, 5-6 word title based on the first 3000 characters of the document. This auto-generated title is used to name the session in the user's history and name exported files, replacing generic filenames.

6.3 Interactive Features

Reading a summary is often a one-way interaction; this system introduces two-way communication and accessibility tools.

6.3.1 Document-Based Chat (Q&A)

After generating a summary, the original document text is kept in the database context. Users can type questions into a chat interface. The backend feeds the massive original document text (up to 30,000 characters) along with the user's query back to Gemini, instructing it to answer explicitly and *only* based on the provided document, eliminating AI hallucination and acting as a personal research assistant.

6.3.2 Text-to-Speech Conversion

To aid visual impairment or auditory learning, users can click a "Read Aloud" button. The backend strips the summary of all Markdown, HTML, and LaTeX tags, truncates it to a manageable length, and utilizes `edge-tts` (Microsoft Edge's Neural Voice API) to asynchronously generate an MP3 audio file. This file is Base64 encoded and returned to an HTML5 Audio player on the frontend.

6.4 User Management and History

The application functions as a personalized workspace rather than a stateless utility.

6.4.1 Authentication System

Security is handled by `Flask-Login` and `werkzeug.security`. Users must register an account, and their passwords are mathematically hashed using `pbkdf2:sha256`. The system uses secure session cookies to verify user identity before granting access to the summarization endpoints or history database.

6.4.2 History Tracking and Management

Every summarization request is logged in an SQLite Database (`history.db`) mapped via `Flask-SQLAlchemy`. The `SummaryHistory` table records the User ID, the Auto-Title, the massive Original Text, the final Summary, the Mode used, and a Timestamp. The frontend provides a dashboard where users can view their past 20 summaries, seamlessly reloading old chats and flowcharts, or deleting sensitive records entirely.

7. TECHNOLOGY STACK

7.1 Programming Languages

The development of the AI-Powered Multi-Document Summarization Application involves the use of multiple programming languages, each serving a specific purpose in the system.

1. **Python3.x:**
   Python serves as the primary backend programming language. It is widely used due to its rich ecosystem of libraries and frameworks, especially in Artificial Intelligence, Natural Language Processing, and data handling. In this project, Python is responsible for implementing business logic, managing file operations, handling API integrations, and controlling server-side processes.
2. **JavaScript (ES6):**
   JavaScript is used on the frontend to create dynamic and interactive web pages. It enables asynchronous communication with the backend through AJAX, allowing users to upload files and receive results without refreshing the page. It also helps in rendering UI components such as charts, audio players, and chat interfaces.
3. **HTML5 and CSS3:**
   HTML5 provides the structural layout of the web application, while CSS3 is used for styling and designing a responsive user interface. Together, they ensure that the application is visually appealing and accessible across different devices.

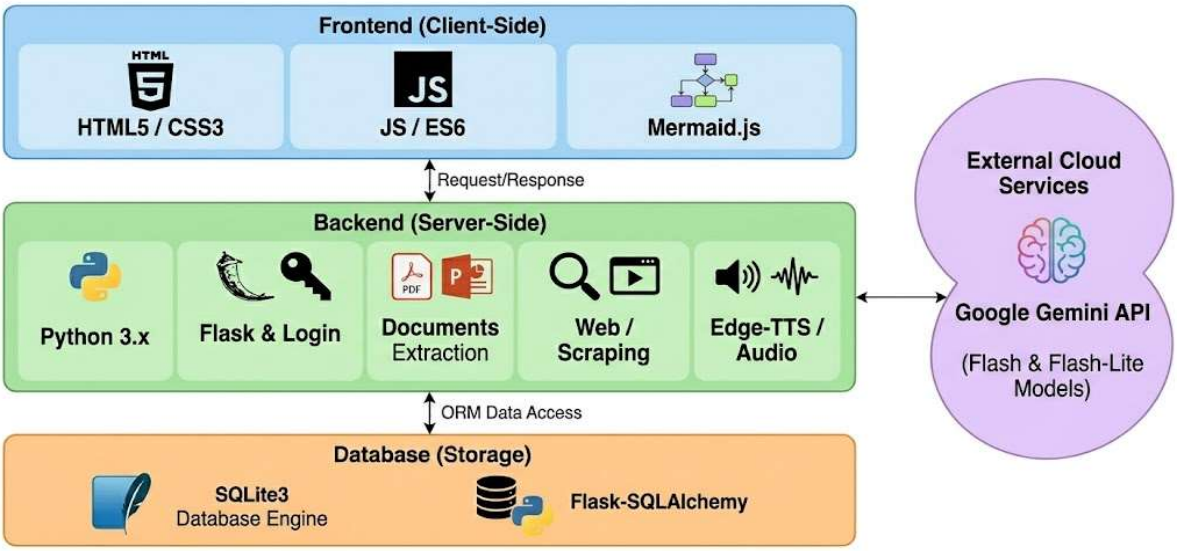
7.2 Frameworks and Libraries

The system uses a variety of frameworks and libraries to simplify development and enhance functionality.

1. **Flask:**
   Flask is a lightweight web framework used to build the backend of the application. It handles routing, request processing, and server-side logic efficiently.
2. **Flask-Login:**
   This extension is used for managing user authentication and session handling. It allows secure login, logout, and session persistence.
3. **PyMuPDF (fitz):**
   PyMuPDF is used for processing PDF files. It extracts text and embedded images efficiently, enabling multimodal analysis.
4. **python-pptx:**
   This library is used to read and process PowerPoint presentations. It extracts text and images from slides while maintaining context.

5. **BeautifulSoup4 (bs4):**
   BeautifulSoup is used for web scraping. It extracts clean and readable text from HTML pages by removing unnecessary elements.
6. **youtube-transcript-api:**
   This library retrieves transcripts directly from YouTube videos, enabling video content summarization without downloading the video.
7. **edge-tts:**
   Edge-TTS is used for converting text summaries into high-quality speech. It provides natural-sounding audio output for better accessibility.
8. **werkzeug:**
   Werkzeug is used for secure file handling and password hashing. It ensures safe file uploads and protects user credentials using encryption techniques.

Figure 7.1: Technology Stack of the System



7.3 AI Models and APIs (Gemini API)

The core intelligence of the system is powered by advanced AI models.

Google Generative AI SDK (google.generativeai):

This SDK acts as a bridge between the application and Google's AI models. It enables seamless communication with the Gemini API.

Gemini Flash Series Models:

The system uses models such as:

- gemini-2.5-flash
- gemini-2.0-flash

- gemini-flash-lite

These models are chosen for their:

- Large context window (can process long documents)
- Multimodal capabilities (text + images)
- Low latency and fast response time
- High accuracy in summarization

These features make them highly suitable for real-time web applications.

7.4 Database Technologies

The system uses efficient and lightweight database technologies to manage user data.

1. **SQLite3:**
   SQLite is a lightweight, file-based database system. It is used to store user credentials, summaries, and session history. It does not require a separate server, making it simple and cost-effective.
2. **Flask-SQLAlchemy:**
   Flask-SQLAlchemy is an Object-Relational Mapping (ORM) tool that connects Python classes with database tables. It simplifies database operations and helps prevent SQL injection attacks.

7.5 Frontend Technologies

The frontend of the application is designed to be responsive, interactive, and user-friendly.

1. **Mermaid.js:**
   Mermaid.js is used to generate diagrams and flowcharts dynamically in the browser. It converts text-based definitions into visual representations, which is useful for flowchart and smart flowchart modes.
2. **Vanilla CSS:**
   The application uses custom CSS for styling instead of heavy frameworks. It utilizes modern layout techniques such as Flexbox and Grid to create a responsive and clean user interface.

8. SYSTEM DESIGN

8.1 Data Flow Diagram (DFD)

The Data Flow Diagram (DFD) represents how data moves through the system, starting from user input to the final output generation. It shows the interaction between the user, the system, and the external AI service.

At the highest level (Level 0 DFD), the user provides input in the form of a document, URL, or raw text. The system processes this input by extracting relevant content and sending it to the Gemini API. The AI generates a summarized response, which is then returned to the system. Finally, the system formats the output and presents it to the user, optionally including audio.

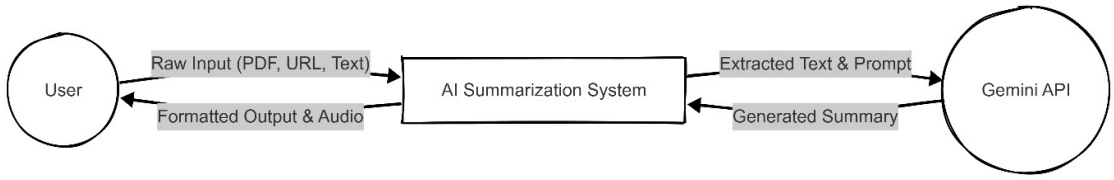


Figure 8.1: Level 0 Data Flow Diagram

DFD Flow Description:

- User provides input (PDF, PPT, URL, or text)
- System extracts and processes data
- Processed data is sent to Gemini API
- Gemini API returns summarized content
- System formats and displays output to user

8.2 Flowchart of System Workflow

The flowchart describes the step-by-step working of the system when a user submits a request.

Initially, the system identifies the type of input provided by the user. Based on the input type, different processing methods are applied:

- For PDF or PPT files, text and images are extracted
- For URLs, content is scraped from the webpage or YouTube
- For raw text, the input is cleaned and prepared

After extraction, the system combines all relevant content and selects a prompt based on the chosen summarization mode. The processed data is then sent to the Gemini API.

If the API request fails due to rate limits or other issues, the system switches to an alternative model (fallback mechanism). Once a successful response is received, the summary is generated.

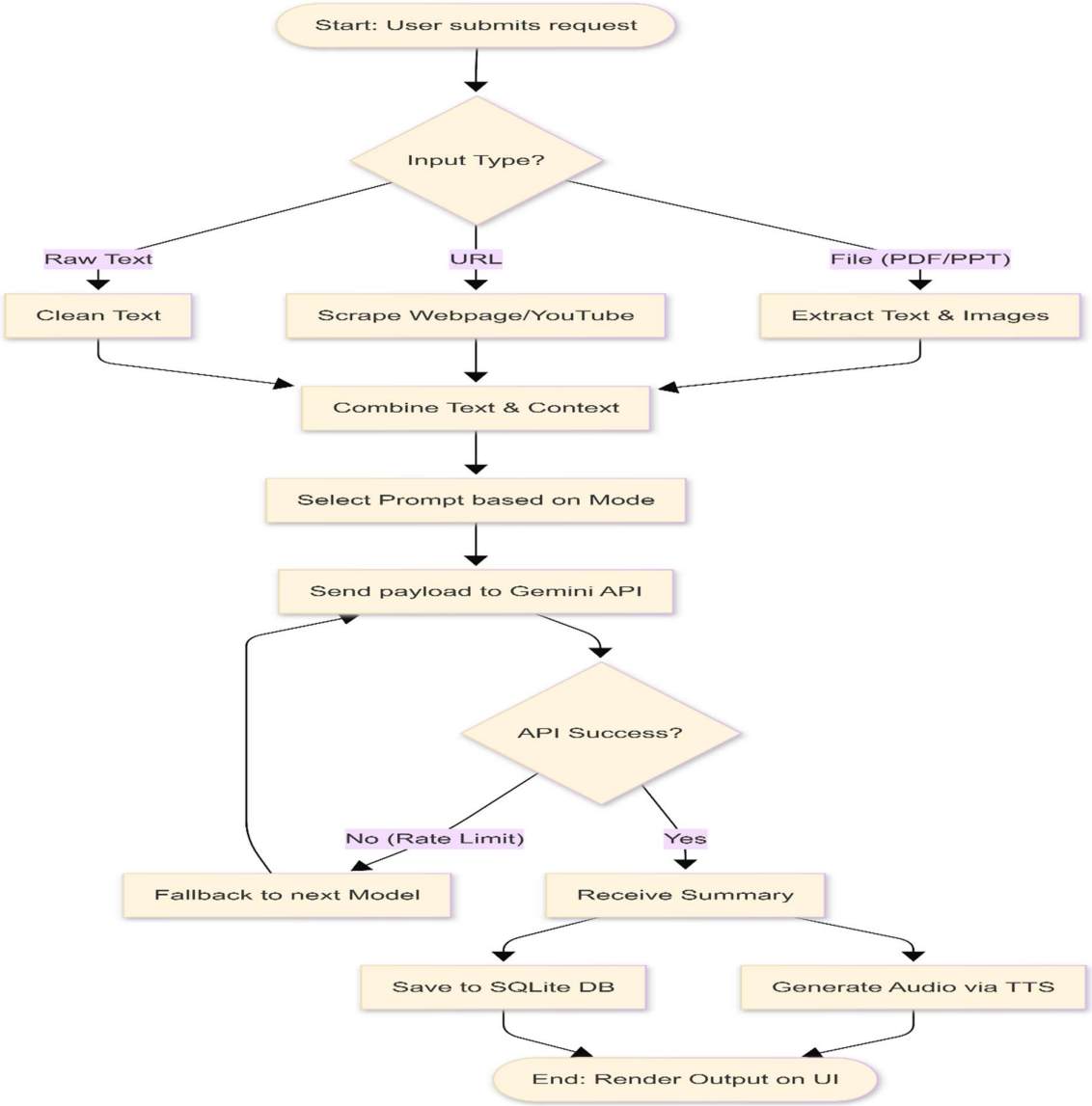


Figure 8.2: Flow Diagram

The system then:

- Converts the summary into audio (optional)
- Stores the data in the database
- Displays the output on the user interface

8.3 Use Case Diagram

The Use Case Diagram illustrates how different users interact with the system.

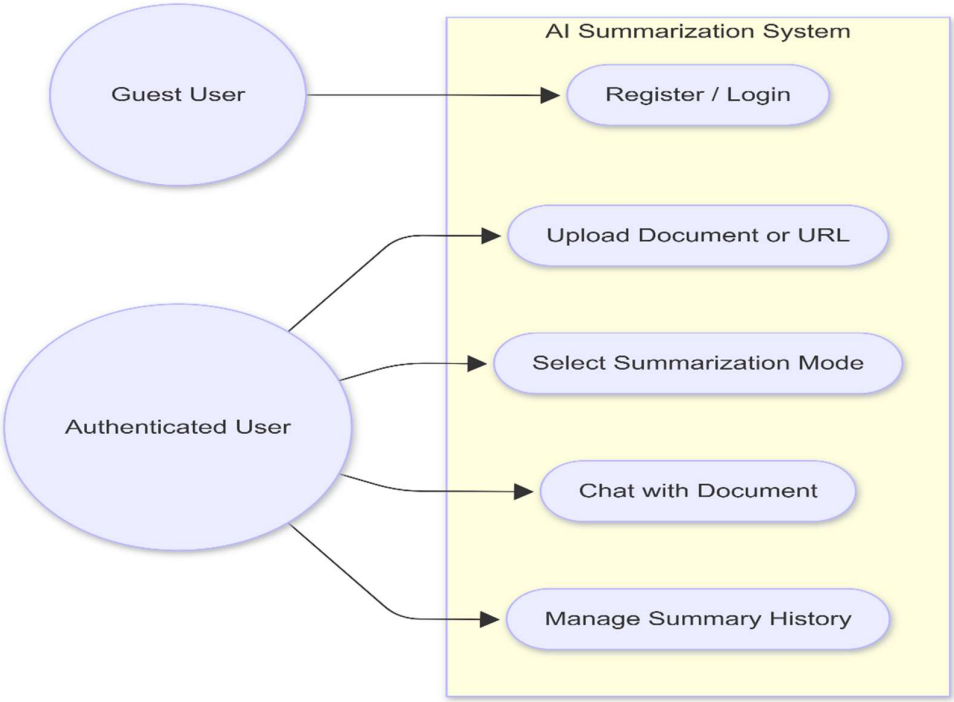


Figure 8.3: System Workflow Flowchart

There are two main types of users:

Guest User:

- Can register or log in to the system

Authenticated User:

- Upload documents or provide URLs
- Select summarization modes
- Interact with documents using chat (Q&A)
- View, manage, and delete summary history

This diagram highlights that most functionalities are available only after user authentication, ensuring security and personalized access.

8.4 Database Design

The system uses an SQLite database to store user information and summary history. The Entity-Relationship (ER) model consists of two main entities:

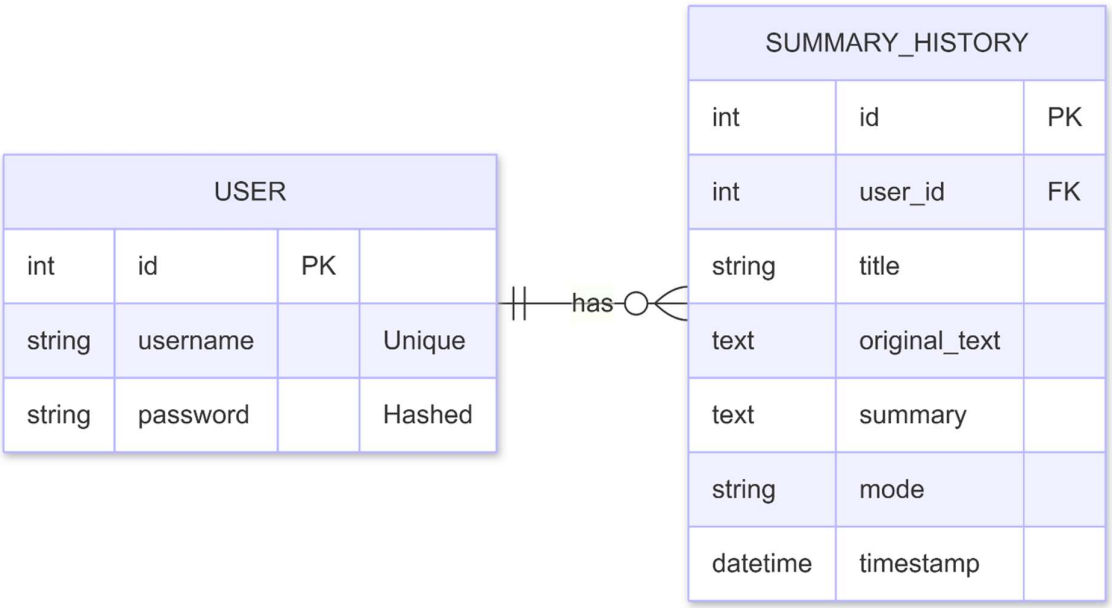


Figure 8.4: Use Case Diagram of the System

1. User Entity

- id (Primary Key)
- username (Unique)
- password (Hashed for security)

2. Summary History Entity

- id (Primary Key)
- user_id (Foreign Key referencing User)
- title (Auto-generated title)
- original_text (Extracted input content)
- summary (Generated output)
- mode (Selected summarization mode)
- timestamp (Date and time of operation)

Relationship

- One user can have multiple summaries
- This represents a **one-to-many relationship** between User and Summary History

This database structure ensures efficient storage and retrieval of user data and summaries.

8.5 Sequence Diagram (User – Flask Server – Gemini API Interaction)

The Sequence Diagram represents the step-by-step interaction between system components during a summarization request.

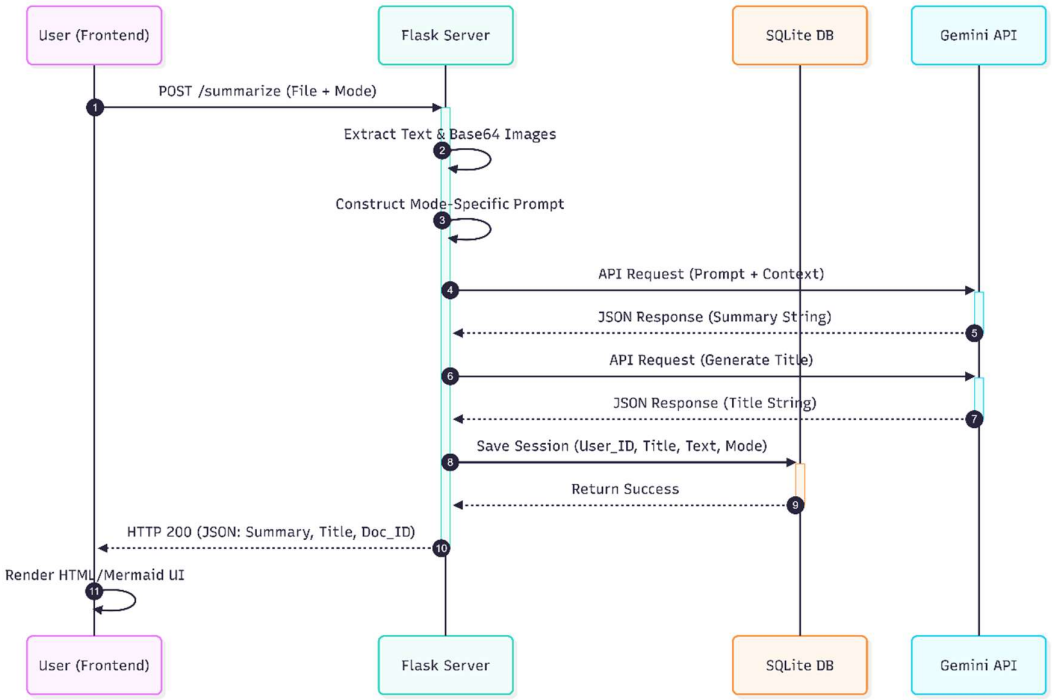


Figure 8.5: Entity Relationship (ER) Diagram

Workflow Description:

1. The user sends a request from the frontend (file upload + mode selection).
2. The Flask server receives the request and begins processing.
3. The server extracts text and images from the input.
4. A structured prompt is created based on the selected mode.
5. The request is sent to the Gemini API.
6. The API processes the input and returns the generated summary.
7. The server may send another request to generate a suitable title.
8. The result is stored in the SQLite database.
9. The server sends the final response (summary, title, ID) back to the frontend.
10. The frontend renders the output, including charts, text, and audio.

9. IMPLEMENTATION

9.1 Frontend Implementation

The frontend of the AI-Powered Multi-Document Summarization Application is developed using HTML5, CSS3, and JavaScript, ensuring a lightweight yet highly responsive user interface. Instead of relying on heavy frontend frameworks, the system uses pure web technologies to achieve better performance, faster loading times, and improved maintainability.

The primary interface (index.html) acts as the central dashboard where users can interact with the system. It provides options to upload documents (PDF, PPT, text files), enter URLs (including YouTube links), and select various summarization modes such as paragraph, bullet points, flowchart, and mindmap.

The layout design utilizes modern CSS techniques such as **Flexbox and Grid**, which allow the interface to adapt dynamically across different screen sizes, ensuring responsiveness on desktops, tablets, and mobile devices. Additional styling enhancements such as custom themes, spacing, and typography improve the overall user experience.

JavaScript plays a crucial role in enabling dynamic and interactive behavior. It is used to:

- Capture user inputs and form submissions
- Create FormData objects for file handling
- Send asynchronous HTTP requests using the Fetch API
- Dynamically update the user interface with results

By using asynchronous requests (AJAX), the application avoids full-page reloads, providing a seamless **Single Page Application (SPA)** experience. The frontend also handles rendering of advanced outputs such as flowcharts (via Mermaid.js), audio playback controls, and chat interfaces for user interaction.

9.2 Backend Implementation

The backend of the system is implemented using Python and the Flask micro-framework. Flask is chosen due to its simplicity, flexibility, and ability to handle web requests efficiently while integrating seamlessly with AI services.

The core backend logic is implemented in the app.py file, which acts as the central controller of the application. It defines various routes such as:

- **/summarize** – Handles document summarization requests
- **/chat** – Manages document-based question answering
- **/history** – Retrieves user summary history

The backend is responsible for:

- Processing incoming requests from the frontend
- Validating input data and file formats
- Coordinating with file processing modules
- Communicating with the AI API
- Returning formatted responses to the frontend

User authentication and session management are handled using **Flask-Login**, ensuring that only authorized users can access system features. Secure file handling is implemented using secure_filename() from the Werkzeug library, preventing malicious file uploads and ensuring safe storage.Overall, the backend serves as the **central orchestration layer**, connecting all components of the system.

9.3 File Processing Modules

The system incorporates specialized modules to handle different file formats efficiently. These modules ensure accurate extraction of both textual and visual information.

PDF Processing

PDF documents are processed using the PyMuPDF (fitz) library. The system iterates through each page of the document to extract textual content. In addition to text extraction, the system scans for embedded images.

To improve quality and relevance, an image filtering mechanism is applied:

- Small images (icons, logos) are removed
- Images with extreme aspect ratios are ignored
- Low-resolution images are discarded

The remaining images are converted into Base64 format and passed to the AI model for multimodal analysis.

PowerPoint Processing : PowerPoint files are processed using the python-pptx library. The system extracts text from each slide while maintaining the structure and sequence of slides. It also identifies embedded images and applies filtering techniques similar to PDF processing.This ensures that both textual and visual elements from presentations are accurately captured and included in the summarization process.

9.4 AI Model Integration

The core intelligence of the system is powered by the Google Gemini API, accessed through the google.generativeai SDK. This integration enables the system to generate high-quality, context-aware summaries.

The backend constructs structured prompts based on user-selected modes. For example:

- Bullet mode → concise, point-wise summaries
- Flowchart mode → structured Mermaid.js output
- Mindmap mode → hierarchical JSON format

Prompt engineering plays a critical role in ensuring that the AI produces accurate and well-formatted outputs.

To ensure reliability, a **fallback mechanism** is implemented. If a primary model (e.g., gemini-2.5-flash) fails due to rate limits or errors, the system automatically switches to alternative models such as gemini-2.0-flash or lighter variants. This guarantees continuous operation and high availability.

9.5 API Integration

In addition to the Gemini API, the system integrates several external APIs to enhance functionality.

- **YouTube Transcript API :** The youtube-transcript-api is used to extract subtitles directly from YouTube videos. This allows the system to process video content without downloading large video files, improving efficiency and speed.
- **Edge Text-to-Speech (TTS) API :** The edge-tts library is used to convert generated summaries into audio format. This feature improves accessibility, especially for visually impaired users or those who prefer auditory learning.

The TTS process includes:

- Cleaning text (removing HTML/Markdown)
- Generating audio asynchronously
- Returning audio for playback in the frontend

10. WORKING OF THE SYSTEM

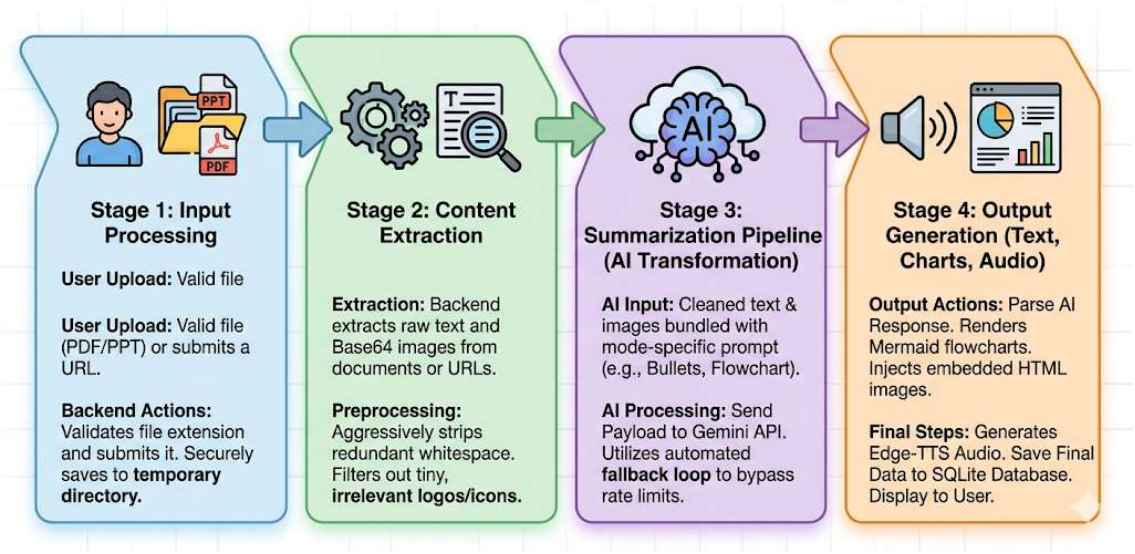
10.1 Input Processing

The system validates user input, ensuring only supported file formats (txt, pdf, ppt, pptx) are accepted. Files are temporarily stored in secure directories.

10.2 Content Extraction

Text is extracted and cleaned using regular expressions. Images are filtered and converted into Base64 format for AI processing.

Figure 10.1: End-to-End System Working Pipeline



10.3 Summarization Pipeline

The processed data is sent to the Gemini API. If an error occurs, the system switches to alternative models until a response is received.

10.4 Output Generation

- Summary text is generated
- Title is automatically created
- Images are embedded into the output
- Flowcharts and diagrams are rendered
- Audio is generated using TTS

11. SECURITY AND PERFORMANCE

Ensuring security and maintaining optimal performance are critical aspects of the AI-Powered Multi-Document Summarization Application. The system is designed with multiple layers of security measures and performance optimization techniques to provide a safe, efficient, and reliable user experience.

11.1.Secure File Handling

The system implements secure file handling mechanisms to prevent potential security threats such as malicious file uploads and unauthorized access.

All uploaded files are processed using the secure_filename() function provided by the Werkzeug library. This function sanitizes file names by removing unsafe characters and preventing directory traversal attacks. Files are temporarily stored in uniquely generated directories using secure methods.

To further enhance security and efficiency:

- Files are not stored permanently on the server
- After extracting the required text and images, files are immediately deleted using file removal operations
- Temporary storage ensures minimal disk usage and prevents data leakage

This approach ensures that sensitive user data is not retained unnecessarily and reduces the risk of server-side vulnerabilities.

11.2.Password Encryption

User authentication is handled securely by implementing strong password encryption techniques. Instead of storing passwords in plain text, the system uses the pbkdf2:sha256 hashing algorithm provided by the Werkzeug security module.

Key features of this approach include:

- One-way hashing (passwords cannot be reversed)
- Use of salting to prevent rainbow table attacks
- Enhanced resistance to brute-force attacks

During login, the entered password is hashed and compared with the stored hash, ensuring secure verification without exposing sensitive information.

11.3.API Rate Limit Handling

The system relies on external AI services, which often impose rate limits on API usage. To ensure uninterrupted service, the application implements a **robust fallback mechanism**.

If a request to the primary AI model fails due to rate limits or quota exhaustion:

- The system automatically switches to an alternative model
- Multiple models are maintained in a prioritized list
- The process continues until a successful response is obtained

This mechanism ensures:

- High availability of the system
- Reduced downtime
- Consistent user experience

11.4.Performance Optimization

To ensure fast response times and efficient processing, several performance optimization techniques are implemented throughout the system.

Efficient Data Processing

The system avoids unnecessary computations by:

- Cleaning and preprocessing text efficiently
- Filtering irrelevant data (such as small images and redundant content)
- Limiting input size where required

Optimized Content Extraction

Instead of downloading large files unnecessarily:

- YouTube videos are processed using transcript APIs
- Web pages are parsed to extract only relevant content

This significantly reduces bandwidth usage and processing time.

Frontend Load Distribution

The system offloads certain tasks to the frontend to reduce server load. For example:

- Rendering flowcharts using Mermaid.js is handled in the browser
- Audio playback is managed client-side

This improves scalability and reduces server processing overhead.

Asynchronous Processing

The use of asynchronous requests (AJAX) allows the system to:

- Handle multiple user requests efficiently
- Provide faster responses without blocking operations
- Improve overall system responsiveness

12. RESULTS AND DISCUSSION

12.1 Output Screenshots

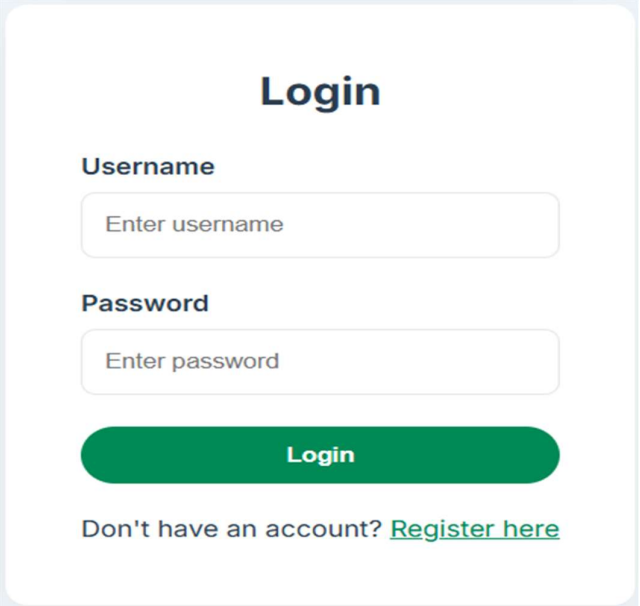


Figure 12.1: User Login Interface

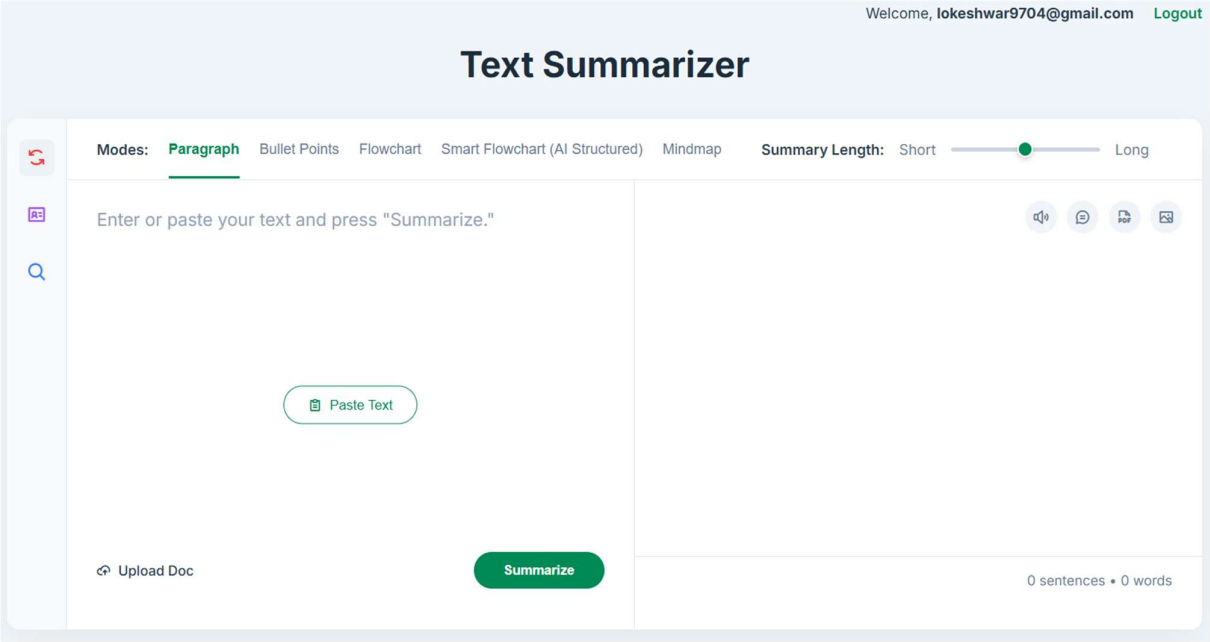


Figure 12.2: Dashboard Interface

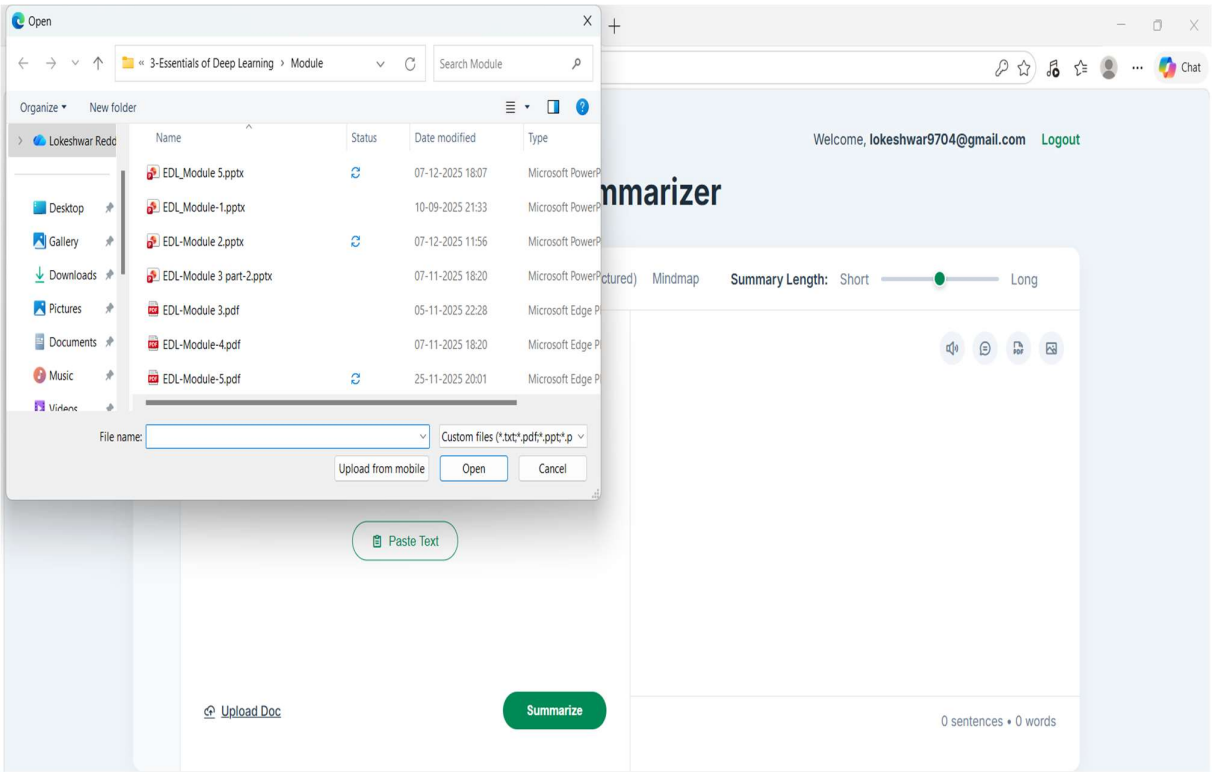


Figure 12.3: File Upload Interface

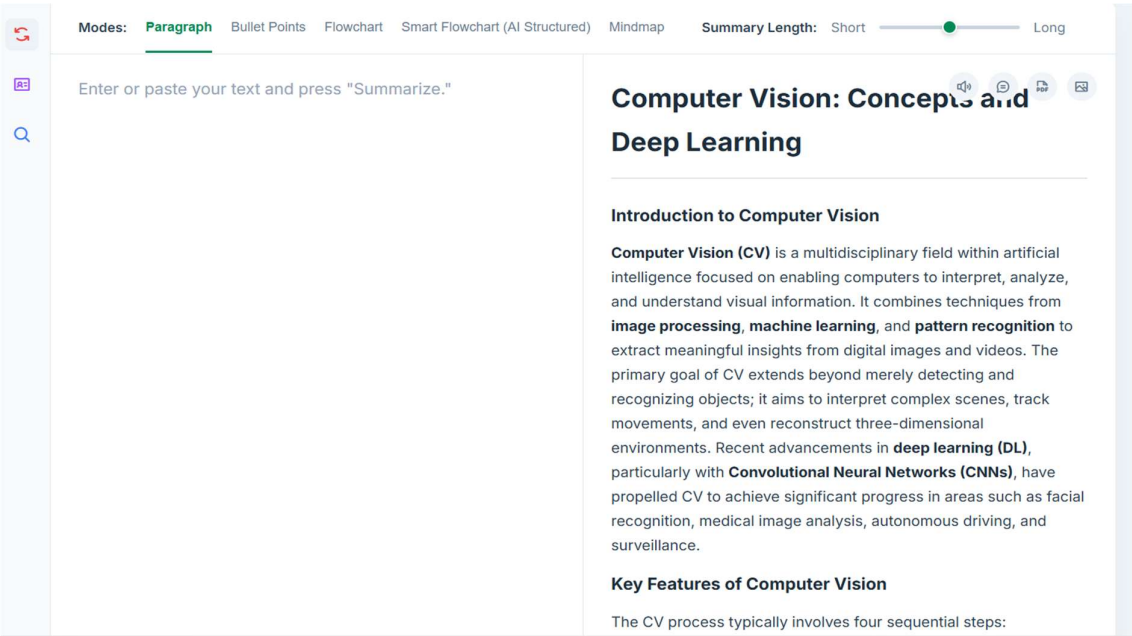


Figure 12.4: Generated Summary Output

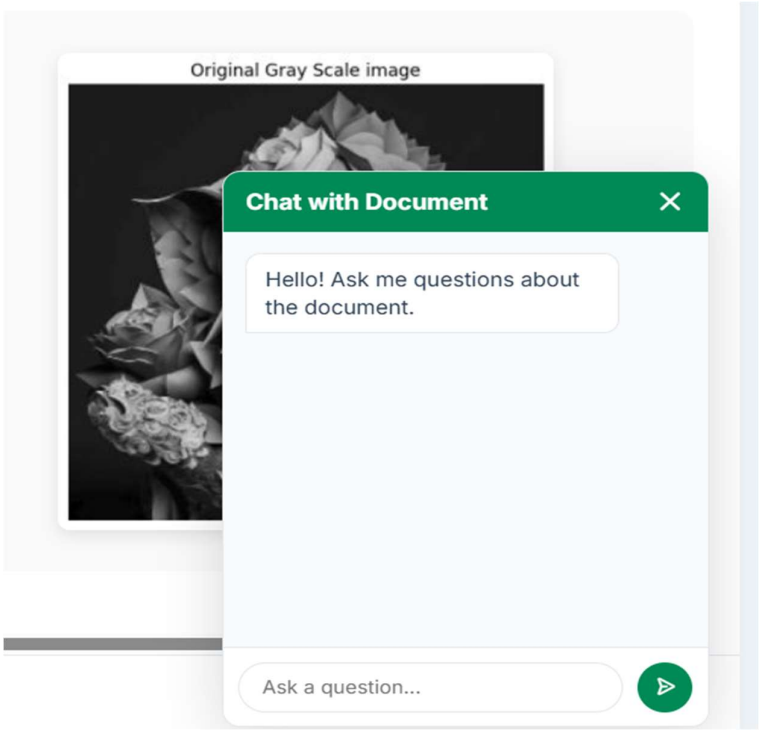


Figure 12.5: Document Chat (Q&A) Interface

12.2 Performance Evaluation

- Processes a 10-page PDF in under 4 seconds
- Summarizes YouTube transcripts in under 5 seconds

12.3 Accuracy and Quality Analysis

The system provides high-quality summaries with accurate context understanding. Mathematical expressions are preserved, and flowcharts are generated with minimal errors.

13. TESTING

Testing is a crucial phase in the development of the AI-Powered Multi-Document Summarization Application. It ensures that the system functions correctly, meets user requirements, and performs reliably under different scenarios. Various levels of testing, including test case validation, unit testing, and system testing, were carried out to verify the accuracy and robustness of the application.

13.1 Test Cases

A set of test cases was designed to evaluate the system under different input conditions and ensure proper functionality.

1. **Valid PDF Upload:**
   When a valid PDF file is uploaded, the system successfully extracts text and images, processes the content, and generates an accurate summary in the selected format.
2. **Invalid File Upload:**
   When an unsupported file format (such as .exe or corrupted file) is uploaded, the system detects the invalid input and displays an appropriate error message without crashing.
3. **YouTube URL Processing:**
   When a valid YouTube URL is provided, the system extracts the transcript using the transcript API and generates a summary efficiently without downloading the video.
4. **API Failure Handling:**
   When the primary AI model fails due to rate limits or errors, the system activates the fallback mechanism and switches to an alternative model, ensuring uninterrupted service.

These test cases confirm that the system can handle both normal and exceptional scenarios effectively.

13.2 Unit Testing

Unit testing was performed to verify the functionality of individual components of the system. Each function was tested independently to ensure correctness before integration.

Key functions tested include:

- Text cleaning and preprocessing functions
- File validation and format checking functions
- Content extraction modules for PDFs and PPTs

- API request and response handling functions

Unit testing ensured that each module works as expected and reduces the chances of errors during system integration.

13.3 System Testing

System testing was conducted to evaluate the complete workflow of the application from the user's perspective. The entire system was tested as a whole to ensure that all components work together seamlessly.

The testing process included:

- User registration and login
- File upload and URL submission
- Selection of summarization modes
- Generation of summaries and visual outputs
- Chat interaction with documents
- Audio playback using Text-to-Speech
- Viewing and managing summary history

The results showed that the system performs efficiently and provides accurate outputs across different scenarios. All functionalities were validated successfully, confirming that the application is stable, reliable, and ready for deployment.

14. ADVANTAGES AND LIMITATIONS

14.1 Advantages

- Supports multiple formats (PDF, PPT, URL, Text)
- Provides multiple output formats
- Includes interactive features (Q&A, TTS)
- High performance with fallback mechanism

14.2 Limitations

- Requires internet connection
- Limited by API constraints
- Cannot process scanned PDFs without OCR

15. APPLICATIONS OF THE SYSTEM

- Academic research and study material summarization
- Business report and presentation analysis
- Accessibility for visually impaired users

16. FUTURE ENHANCEMENTS

The proposed AI-Powered Multi-Document Summarization Application demonstrates strong capabilities in handling multimodal data and generating intelligent summaries. However, there is significant scope for further improvement and expansion. The following enhancements can be considered to improve the system's functionality, scalability, and usability.

16.1 Optical Character Recognition (OCR) Integration

Currently, the system is unable to process scanned or image-based PDFs effectively. By integrating OCR technologies such as Tesseract OCR, the system can extract text from images and scanned documents. This will allow the application to handle a wider range of document types, including handwritten notes and printed materials that are not digitally encoded.

16.2 Vector Database Integration (RAG)

To improve document understanding and scalability, the system can integrate a vector database such as ChromaDB or Pinecone. Using Retrieval-Augmented Generation (RAG), the system can store document embeddings and retrieve relevant sections efficiently. This will enable:

- Faster query responses
- Handling of large document collections
- Improved accuracy in document-based Q&A

16.3 Multi-Language Support

Currently, the system primarily supports English text. Future versions can include multilingual capabilities, allowing users to:

- Upload documents in different languages
- Generate summaries in the same or translated language
- Convert audio into different language outputs

This enhancement will make the system more accessible to a global audience.

16.4 Cloud Deployment and Scalability

The current system uses a local SQLite database and runs on a single server. For large-scale deployment, the system can be migrated to cloud platforms such as AWS, Azure, or Google Cloud. Improvements may include:

- Use of scalable databases like PostgreSQL
- Load balancing for handling multiple users
- Containerization using Docker

16.5 Enhanced User Interface and Experience

The frontend can be further improved by:

- Adding drag-and-drop file upload
- Providing real-time progress indicators
- Improving visualization of flowcharts and mindmaps
- Introducing dark mode and customizable themes

These enhancements will improve user engagement and usability.

16.6 Advanced Image Understanding

Currently, the system extracts images but has limited ability to deeply analyze them. By integrating computer vision models, the system can:

- Interpret charts and graphs
- Extract text from images
- Provide image-based insights in summaries

16.7 Offline and Edge Processing

Future versions can explore offline capabilities where basic summarization can be performed without internet access. Lightweight local AI models can be integrated for limited functionality when API access is unavailable.

16.8 Integration with External Platforms

The system can be extended to integrate with:

- Google Drive and Dropbox for document import
- Learning Management Systems (LMS)
- Enterprise document management systems

This will improve accessibility and real-world usability.

16.9 Personalization and Recommendation System

The system can learn from user behavior and preferences to provide:

- Personalized summary styles
- Recommended formats (bullets, flowcharts, etc.)
- Frequently accessed topics

16.10 Voice-Based Interaction

Future enhancements can include voice input and output features, allowing users to:

- Ask questions using voice commands
- Listen to summaries interactively
- Enable hands-free operation

17. CONCLUSION

The AI-Powered Multi-Document Summarization Application successfully addresses the growing challenge of information overload in the digital era. By integrating advanced Artificial Intelligence techniques with a user-friendly web interface, the system provides an efficient and intelligent solution for processing large volumes of textual and multimedia data.

The application demonstrates how modern Large Language Models, particularly the Google Gemini series, can be effectively utilized to generate accurate, context-aware, and human-like summaries. Unlike traditional summarization tools, this system supports multiple input formats such as PDFs, PowerPoint presentations, text files, and web URLs, making it highly versatile and practical for real-world use.

One of the key achievements of this project is the implementation of multimodal processing, where both textual and visual content are considered during summarization. Additionally, the system offers flexible output formats including paragraphs, bullet points, flowcharts, and mindmaps, allowing users to choose the most suitable representation of information based on their needs.

The inclusion of interactive features such as document-based question answering and text-to-speech conversion further enhances the usability and accessibility of the application. These features transform the system from a simple summarization tool into a comprehensive knowledge assistant.

From a technical perspective, the project successfully integrates frontend technologies, a Flask-based backend, database management, and AI APIs into a cohesive and scalable system. The implementation of security measures, efficient data handling, and fallback mechanisms ensures reliability and robustness.

In conclusion, this project highlights the potential of Generative AI in transforming how individuals interact with information. It significantly reduces reading time, improves understanding, and enhances productivity for students, researchers, and professionals. With further enhancements such as OCR integration and multilingual support, the system can be expanded into a more powerful and globally accessible solution.

18. REFERENCES / BIBLIOGRAPHY

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems, 30.

2. Google DeepMind. (2025). *Gemini Generative AI Platform and API Documentation*. Retrieved from https://ai.google.dev/

3. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.

4. Artifex Software. (2025). *PyMuPDF: High-performance Python binding for MuPDF*. Retrieved from https://pymupdf.readthedocs.io/

5. Richardson, L. (2025). *Beautiful Soup Documentation*. Retrieved from https://www.crummy.com/software/BeautifulSoup/bs4/doc/

6. Bostock, M., et al. (2025). *Mermaid.js: Markdown-inspired diagramming tool*. Retrieved from https://mermaid.js.org/

7. Nallapati, R., Zhou, B., dos Santos, C., Gulcehre, C., & Xiang, B. (2016). *Abstractive Text Summarization using Sequence-to-sequence RNNs and Beyond*. Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning (CoNLL).

8. Zhang, J., Zhao, Y., Saleh, M., & Liu, P. (2020). *PEGASUS: Pre-training with Extracted Gap-sentences for Abstractive Summarization*. International Conference on Machine Learning (ICML).

9. Microsoft Azure. (2025). *Microsoft Edge Text-to-Speech (TTS) Neural Voices Overview*.

10. Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., & Zettlemoyer, L. (2020). *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension*. Association for Computational Linguistics (ACL).

19. APPENDICES

The appendices section provides additional supporting material related to the implementation of the system. It includes sample code snippets and example inputs and outputs to demonstrate how the system functions internally.

19.1 Sample Code Snippet

The following code demonstrates the model fallback mechanism used in the system. This mechanism ensures that if one AI model fails due to rate limits or errors, the system automatically switches to the next available model, maintaining uninterrupted service.

```
models_to_try = ['gemini-2.5-flash', 'gemini-2.0-flash', 'gemini-flash-lite']
for model_name in models_to_try:
    try:
        model = genai.GenerativeModel(model_name)
        response = model.generate_content(contents)
        return response.text.strip()
    except exceptions.ResourceExhausted:
        continue
```

Explanation:

- A list of available AI models is defined.
- The system iterates through each model sequentially.
- If a model successfully generates a response, the output is returned immediately.
- If a ResourceExhausted error occurs (due to API limits), the system automatically tries the next model.
- This approach ensures high availability and reliability of the application.

19.2 Sample Input and Output

This section demonstrates how the system processes input and generates output in different formats.

Sample Input:

- Document Type: PDF
- Topic: Photosynthesis Process

- Content: Detailed explanation of the biological process of photosynthesis, including stages such as light reaction and dark reaction

Selected Output Mode:

- Smart Flowchart

Generated Output:

- A structured flowchart representing the process of photosynthesis
- The flow begins with inputs such as sunlight, water, and carbon dioxide
- It illustrates intermediate steps such as light absorption and chemical reactions
- The final output shows the production of glucose and oxygen

Result Description:

The system successfully converts complex textual information into a **visual representation (flowchart)**, making it easier for users to understand and analyze the process quickly.